

算法设计与分析

Design & Analysis of Algorithms

第4章 贪心算法

学习要点

- 理解贪心算法的概念
- 掌握贪心算法的基本要素
 - (1) 贪心策略 (2) 正确性证明
- 掌握设计贪心算法的步骤
- 通过应用范例学习贪心算法设计策略
 - (1) 货币兑付问题
 - (2) 部分背包问题
 - (3) 活动安排问题
 - (4) 最优装载问题
 - (5) 最大子段和
 - (6) 多机调度问题
 - (7) 货郎担问题

4.1 贪心法设计方法及基本要素

贪心：追求利益最大化，或代价（耗费）最小化

例4.1 货币兑付问题

假定出纳员手中有 n 张面值为 $p_i (1 \leq i \leq n)$ 的货币，用集合 $P = \{p_1, p_2, \dots, p_n\}$ 表示这些货币。如果出纳员需支付的现金为 A ，要求出纳员用最少的货币张数支付现金。

形式化： 假设最后的支付集合为 S ，定义 x_i 为：

$$x_i = \begin{cases} 1 & p_i \in S \\ 0 & p_i \notin S \end{cases}$$

问题转换为：

目标函数

$$\dots \min \sum_{i=1}^n x_i \quad \text{且} \quad \sum_{i=1}^n p_i x_i = A$$

约束条件

4.1 贪心法设计方法及基本要素

例4.1 货币兑付问题

■ **最优问题**，在问题的解空间中搜索满足约束条件且使目标函数达到极值的解向量。其中满足约束条件的解称为问题的可行解，使目标函数取极值的可行解，称为最优解。

■ 例4.1中问题的解空间：

$$\{x_1, x_2, \dots, x_n\} \quad x_i = 1, 0$$

■ 共 2^n 个不同的向量，可用穷举法求解。

4.1 贪心法设计方法及基本要素

例4.2 货币兑付问题：用贪心法求解货币兑付问题。设支付现金 $A=25.9$ 元，支付集合 $P=\{10, 5, 1, 0.5, 0.2, 0.1\}$ ，每种各10张

■ 贪心思想：贪心选择货币面值大者。

■ 过程：

$s = \phi$	
$s = \{10\}$	15.9
$s = \{10, 10\}$	5.9
$s = \{10, 10, 5, 0.5, 0.2, 0.2\}$	0

4.1 贪心法设计方法及基本要素

- **贪心法过程**：通过一系列选择得到问题的解。其所做出的每一个选择都是当前状态下的局部最好选择，即贪心选择。贪心法并不总能得到问题的最优解。
- **两个重要性质**：贪心选择性质，最优子结构性质
 - **贪心选择性质**：是指所求问题的整体最优解可以通过一系列局部最优的选择（仅依赖于以往所做过的选择，不依赖于将来所做的选择），即贪心选择来达到。每做出一次贪心选择将所求问题简化为规模更小的子问题。
 - **最优子结构性质**：当一个问题最优解包含其子问题的最优解时，称此问题具有最优子结构性质。

4.2 部分背包问题

■ 问题描述:

输入

- n 个物品组成的集合 O ，每个物品有两个属性 v_i 和 p_i ，分别表示体积和价格
- 背包容量为 C

输出

- 求解一个解决方案 $S = \{x_i | 1 \leq i \leq n, 0 \leq x_i \leq 1\}$ ，使得：

$$\max \sum_{x_i \in S} x_i \cdot p_i$$

优化目标

$$s. t. \sum_{x_i \in S} x_i \cdot v_i \leq C$$

约束条件

选取物品的比例

4.2 部分背包问题

至少有三种看似合理的贪心策略：

(1) 选择**价值最大**的物品，尽可能快地增加背包的总价值。但背包容量却可能消耗得太快，装入背包的物品个数减少，不能保证目标函数达到最大。

(2) 选择**重量最轻**的物品，装入尽可能多的物品，增加背包的总价值。虽然每一步选择使背包的容量消耗得慢了，背包的价值却没能迅速增长，不能保证目标函数达到最大。

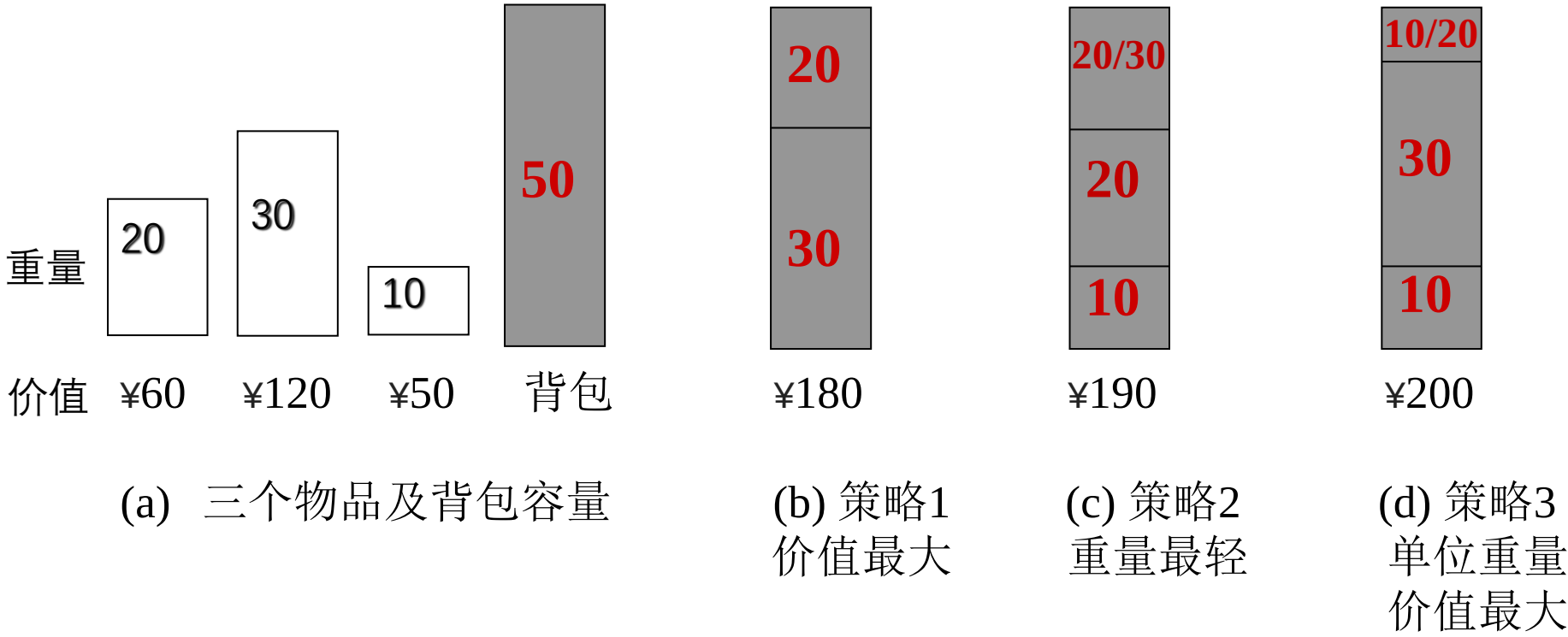
(3) 选择**单位重量价值最大**的物品，在背包价值增长和背包容量消耗两者之间寻找平衡。

最优子结构性质 贪心策略 (3)，每次选择单位重量价值最大的物品，其重量小于背包容量时装入，并将背包容量减去该物品的重量。

最优子问题——它同样是背包问题，只是背包容量减少，物品集合减少。

4.2 部分背包问题

例如，有3个物品，其重量分别是{20, 30, 10}，价值分别为{60, 120, 50}，背包的容量为50，应用三种贪心策略装入背包的物品和获得的价值如图所示。



4.2 部分背包问题

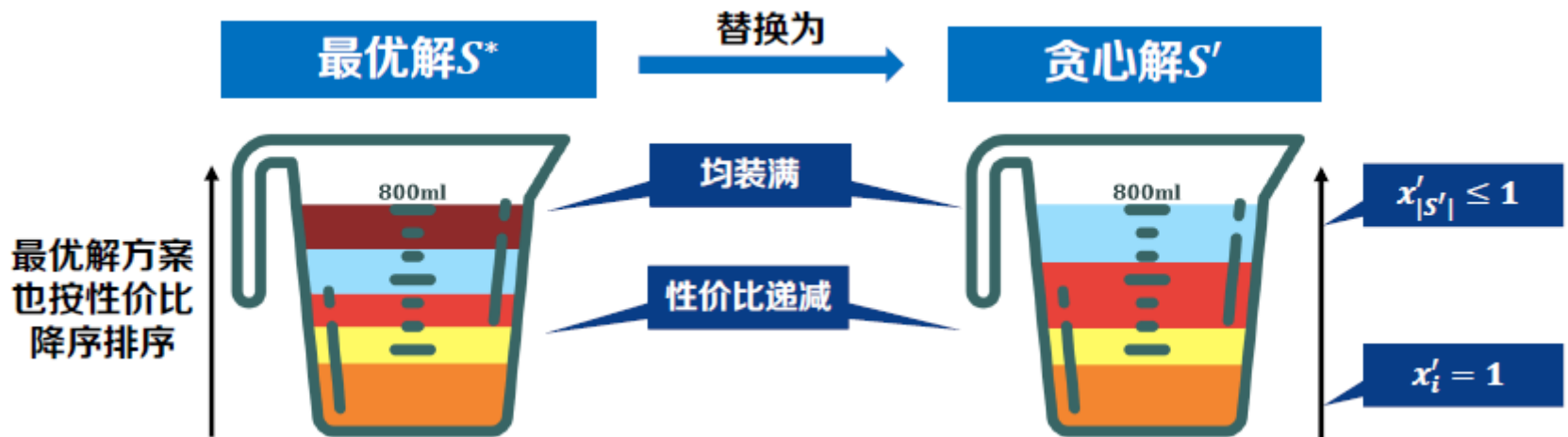
- 容量为**800ml**的杯子，可任选不超过体积上限的饮料进行混合
- 调制饮品价格为各所使用饮料的价格之和，所得饮品价格之和最高

■ 贪心策略：最高性价比优先

性价比=价格/体积

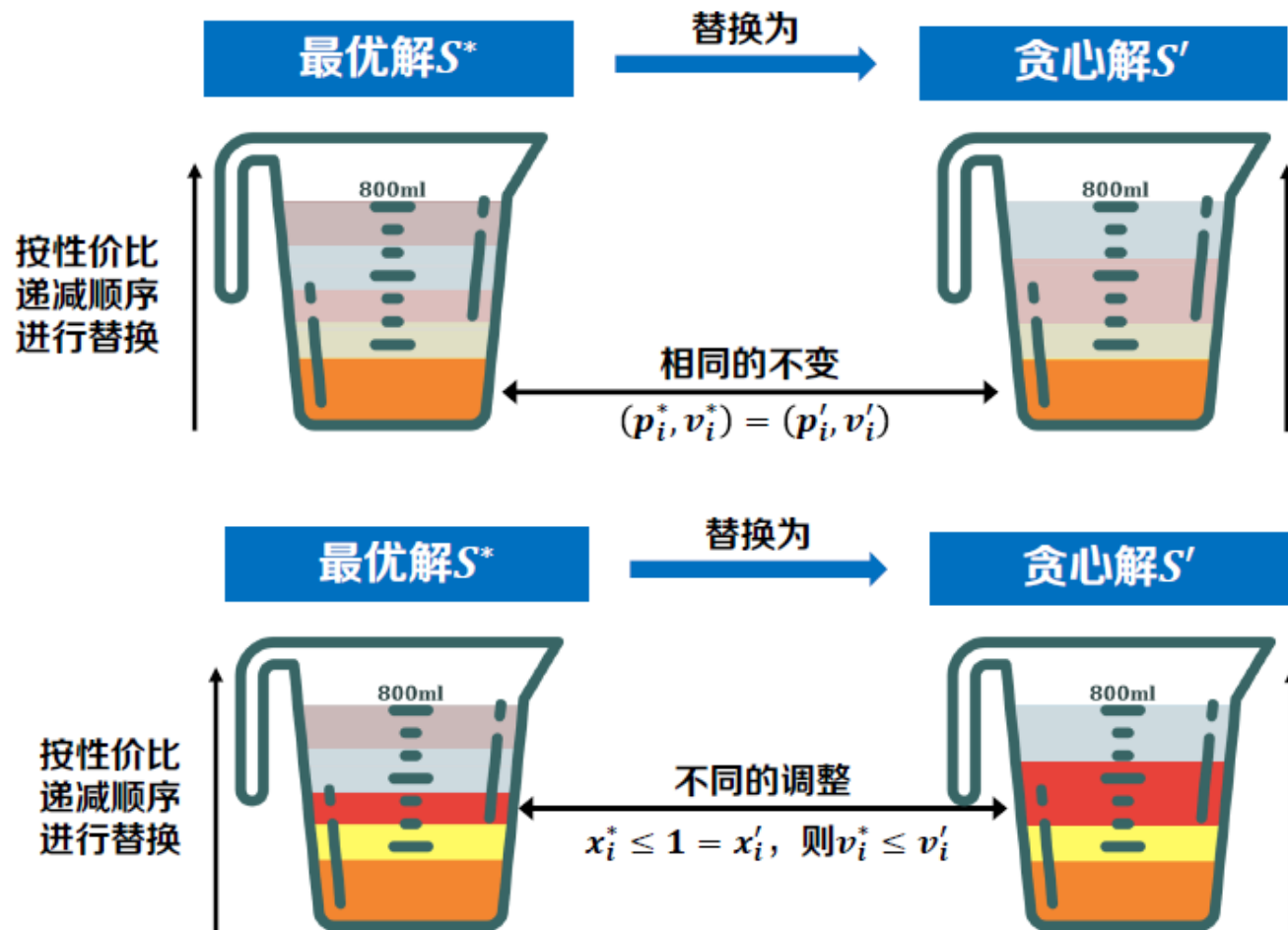
优先选择高性价比的物品放入，尽可能装满背包。

■ 正确性证明：贪心解不劣于最优解



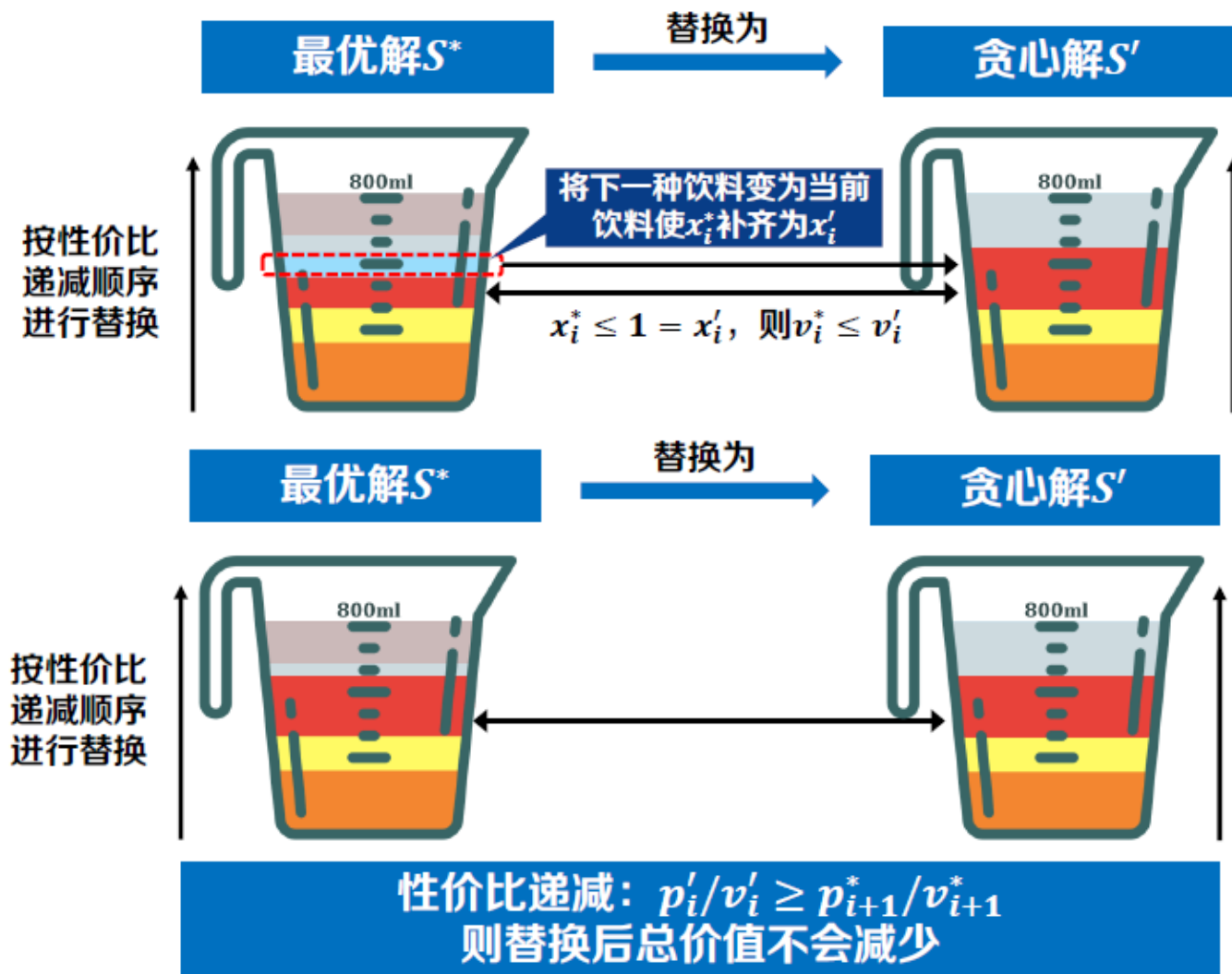
4.2 部分背包问题

■ 正确性证明：贪心解不劣于最优解



4.2 部分背包问题

■ 正确性证明：贪心解不劣于最优解



■ 替换后单位体积价值均不减少，故贪心解不劣于最优解

4.2 部分背包问题

■ 算法伪代码

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的最大值, 最优解方案

计算商品性价比 $Ratio[1..n]$ 并按降序排序

// $Ratio[i], p[i], v[i]$ 分别表示性价比第 i 大的商品的性价比、价格和体积

$i \leftarrow 1$

$ans \leftarrow 0$

// 根据贪心策略求解

while $C > 0$ and $i \leq n$ do

 if $v[i] \leq C$ then

 选择商品 i

$ans \leftarrow ans + p[i]$

$C \leftarrow C - v[i]$

 end

 else

 选择 C 体积的商品 i

$ans \leftarrow ans + p[i] \cdot \frac{C}{v[i]}$

$C \leftarrow 0$

 end

$i \leftarrow i + 1$

end

return ans

$O(n \log n)$

$O(n)$

■ 时间复杂度: $O(n \log n)$

实验4-1： 部分背包问题(贪心算法)

- 问题描述：在选择物品 i 装入背包时，可以选择物品 i 的一部分，而不一定要全部装入背包， $1 \leq i \leq n$ 。

- 示例 1:

输入：

//假设已按单位价值降序排序

```
float profit[5] = { 0, 10, 16, 6, 1};
```

```
float weight[5] = { 0, 3, 5, 2, 1};
```

输出：

- 要求：

1 填空完成程序；

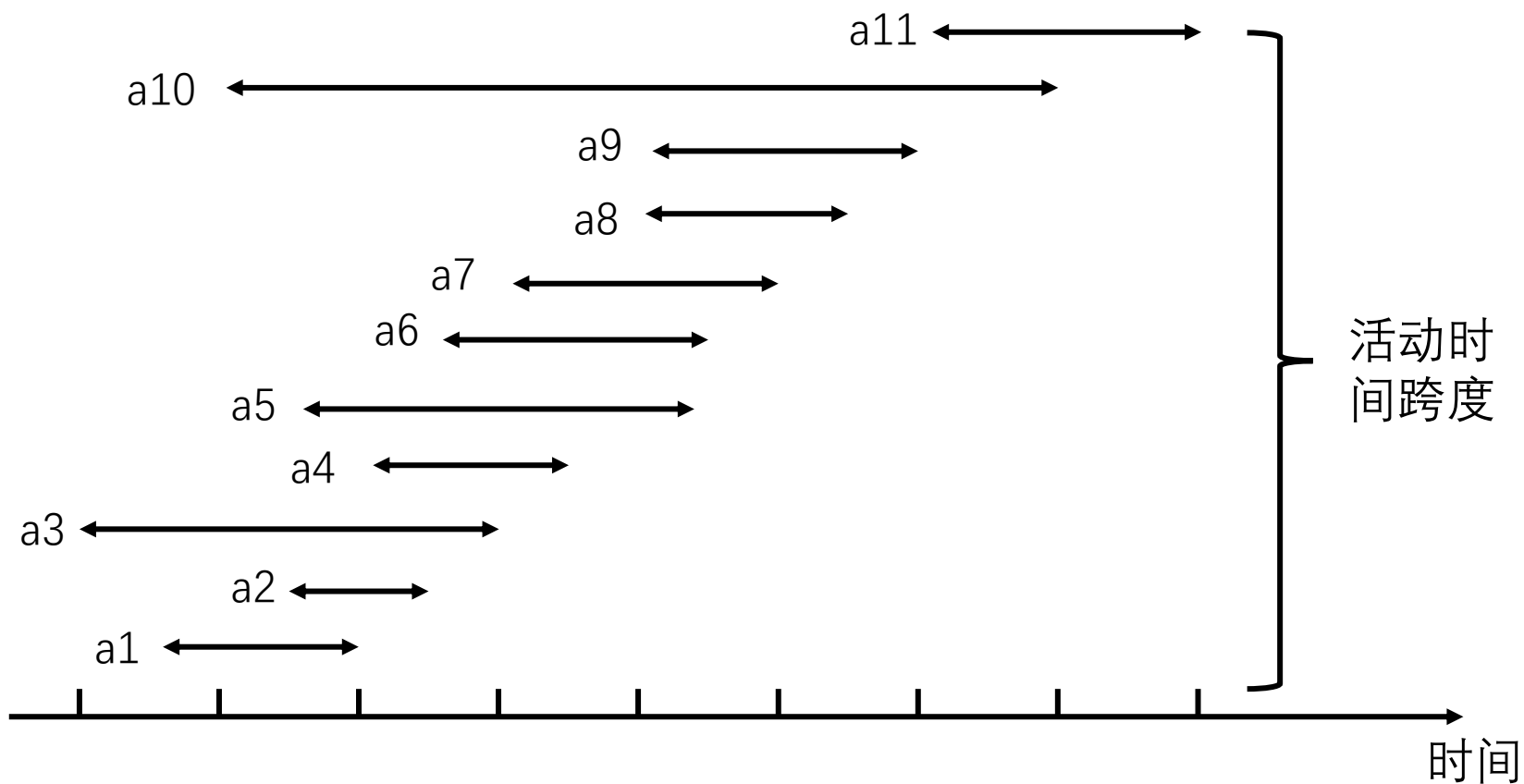
2 输出所选物品重量及价值，总重量与价值。

4.3 活动安排

- **问题描述：** 活动安排问题就是要在所给的活动集合中选出最大的相容活动子集合。
- **形式化：** 设有 n 个活动的集合 $E=\{1,2,\dots,n\}$ ，其中每个活动都要求使用同一资源，在同一时间内只有一个活动能使用这一资源。每个活动 i 都有一个要求使用该资源的起始时间 s_i 和一个结束时间 f_i ，且 $s_i < f_i$ 。如果选择了活动 i ，则它在半开时间区间 $[s_i, f_i)$ 内占用资源。若区间 $[s_i, f_i)$ 与区间 $[s_j, f_j)$ 不相交，则称活动 i 与活动 j 是相容的。也就是说，当 $s_i \geq f_j$ 或 $s_j \geq f_i$ 时，活动 i 与活动 j 相容。
- **实例**

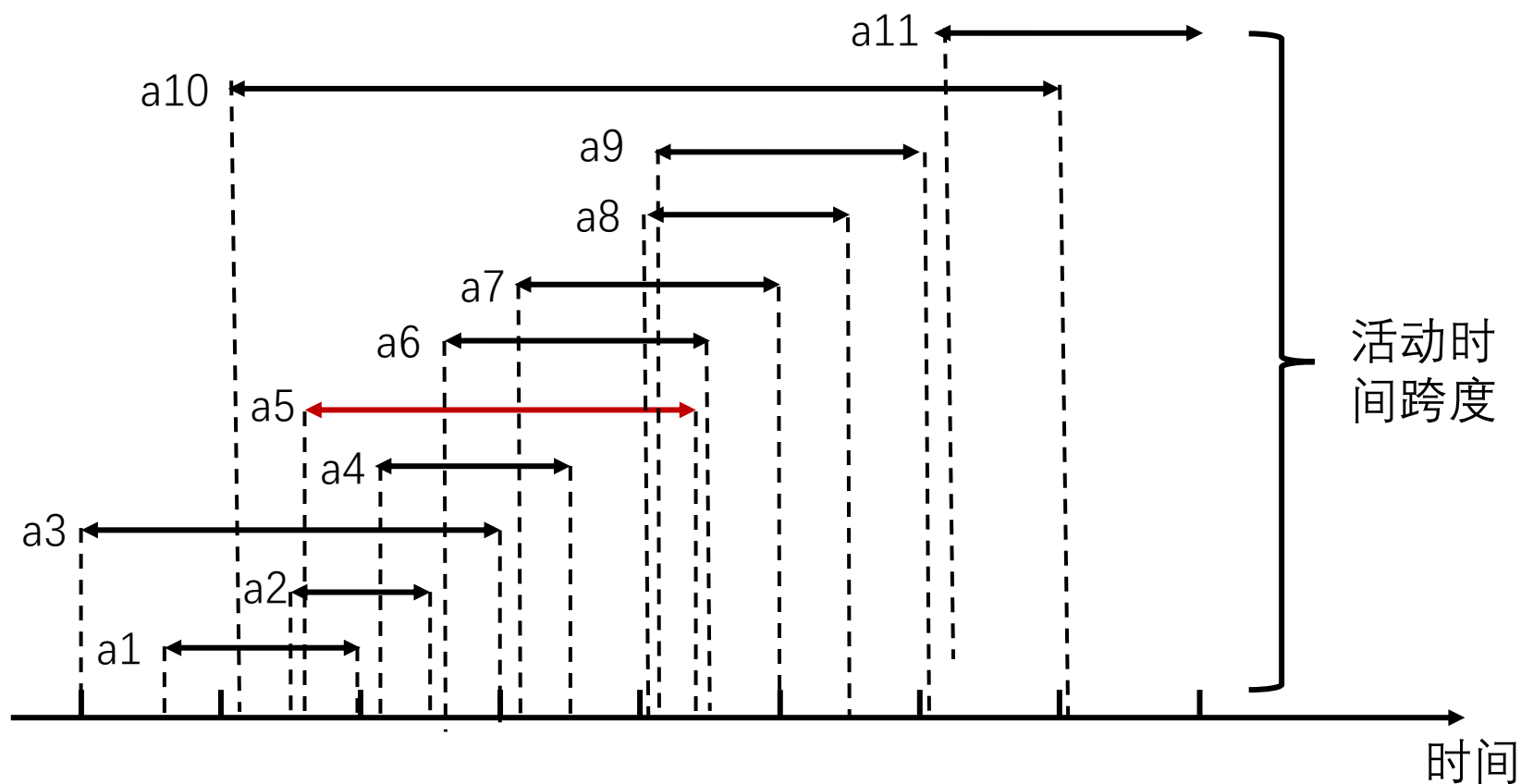
4.3 活动安排

■ 活动分布



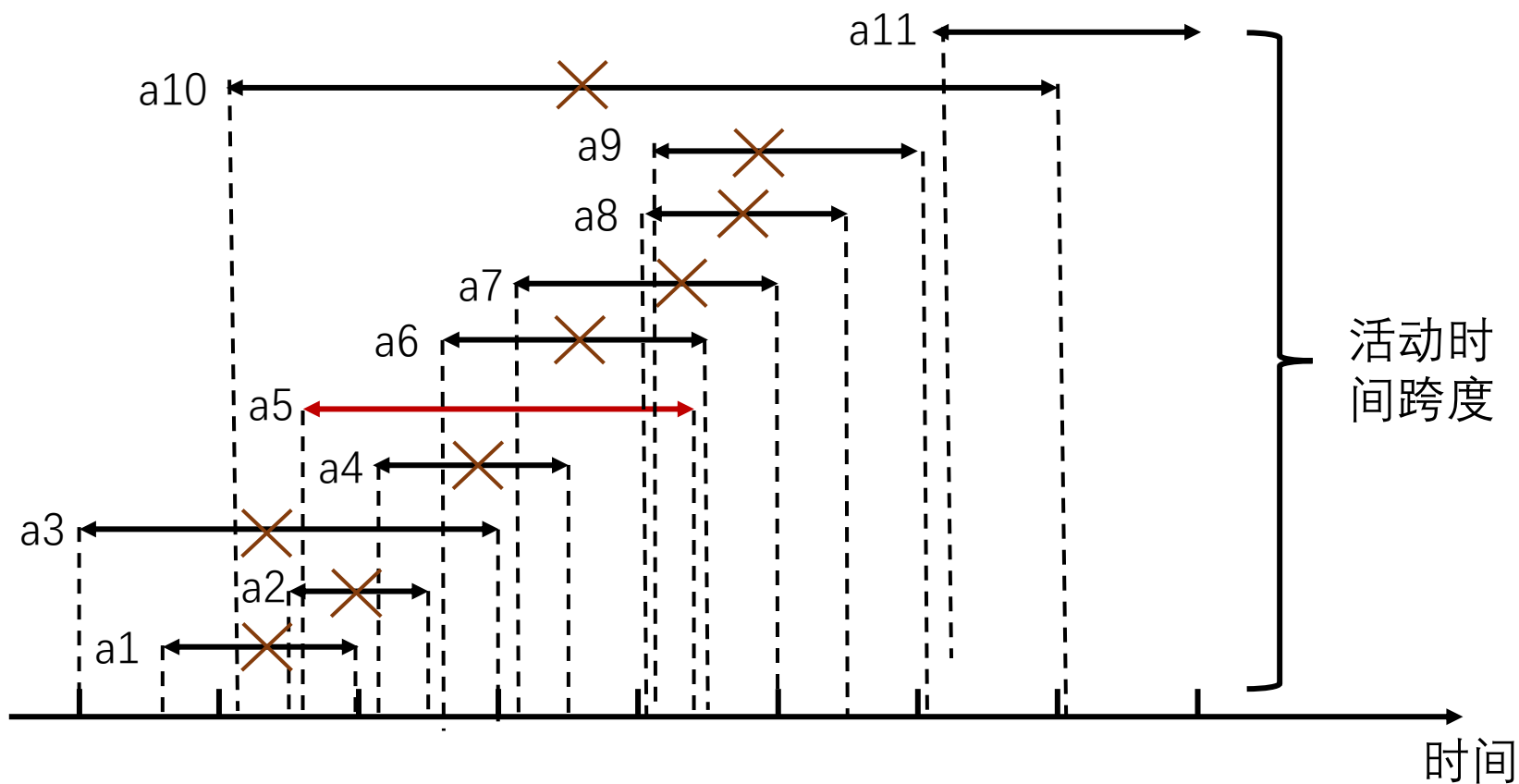
4.3 活动安排

- 活动时间不能冲突



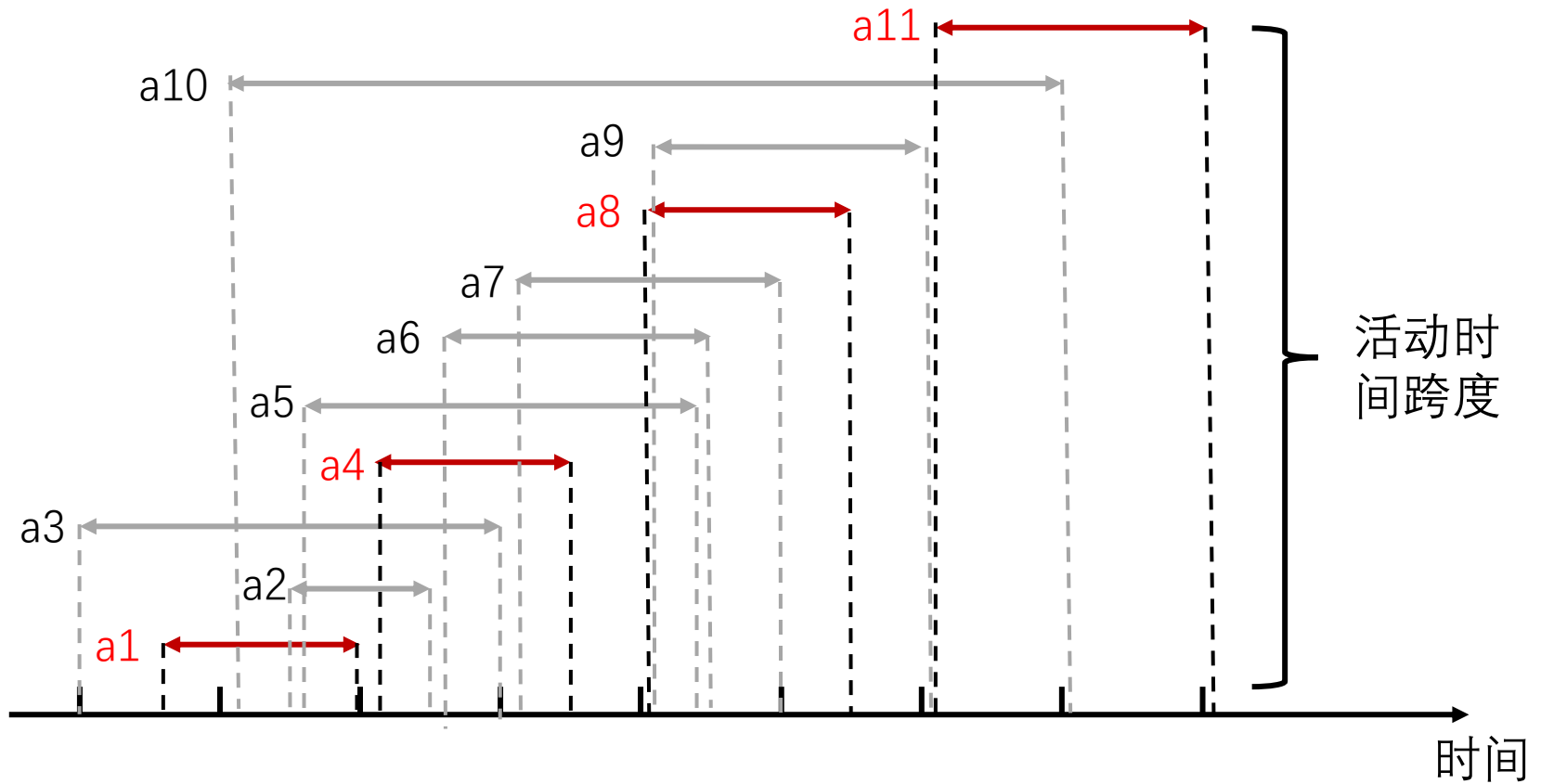
4.3 活动安排

- 活动时间不能冲突



4.3 活动安排

- 活动时间不能冲突，选择更多的活动



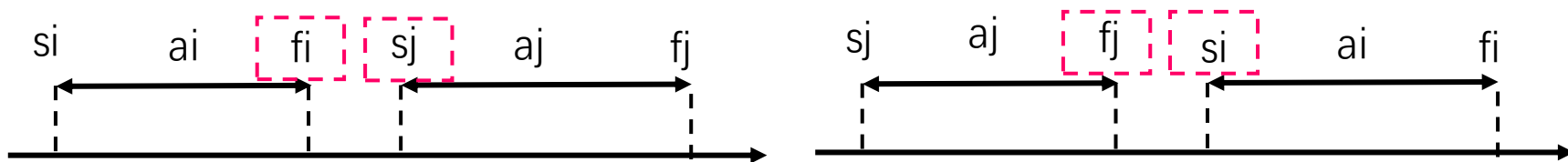
4.3 活动安排

■ 输入

n 个活动组成的集合 $A=\{a_1, a_2, \dots, a_n\}$, 每个活动 a_i 的开始时间 s_i 和结束时间 f_i

■ 输出

找出活动集合 A 的子集 A' , 令 $\max |A'|$ | s.t. 所有 a_i, a_j 满足 $s_i \geq f_j$ 或 $s_j \geq f_i$

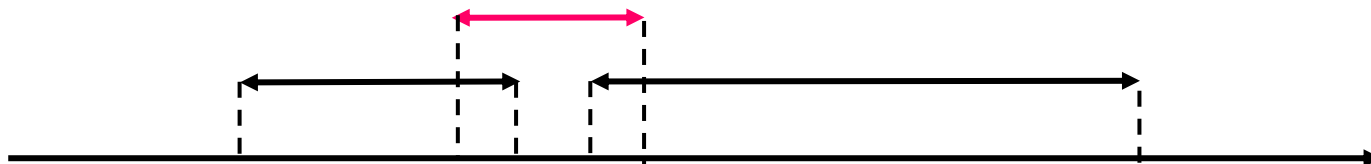


■ 一般步骤

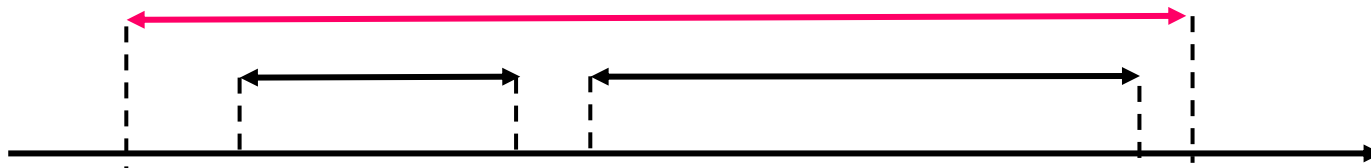
- 提出贪心策略 观察问题特征, 构造贪心选择
- 证明策略正确 假设最优方案, 通过替换证明

4.3 活动安排 贪心策略

- 策略1: 最短活动优先 反例



- 策略2: 最早开始优先 反例

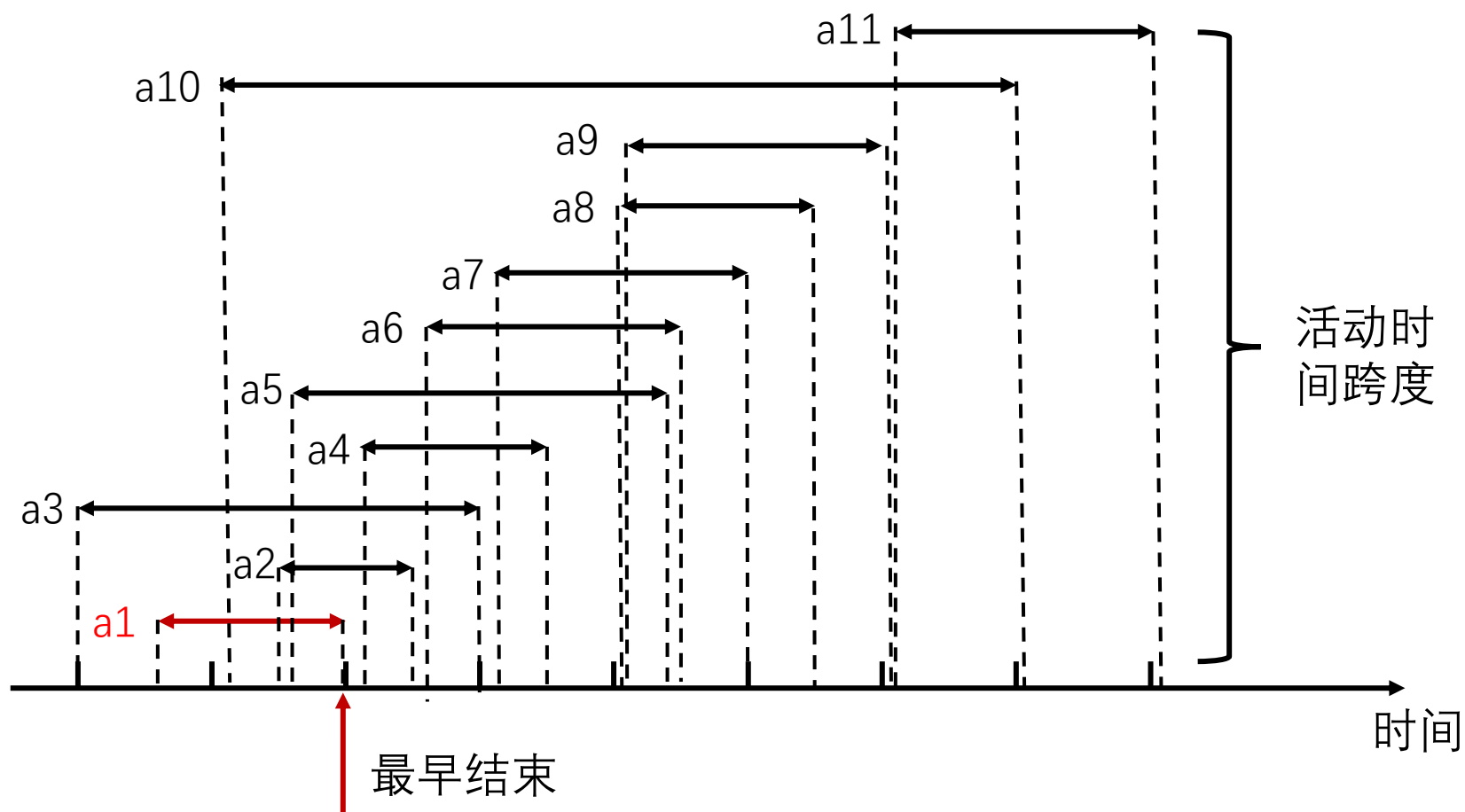


- 最早结束活动优先

选择最早结束的活动，可以给后面的活动留更大的选择空间

4.3 活动安排

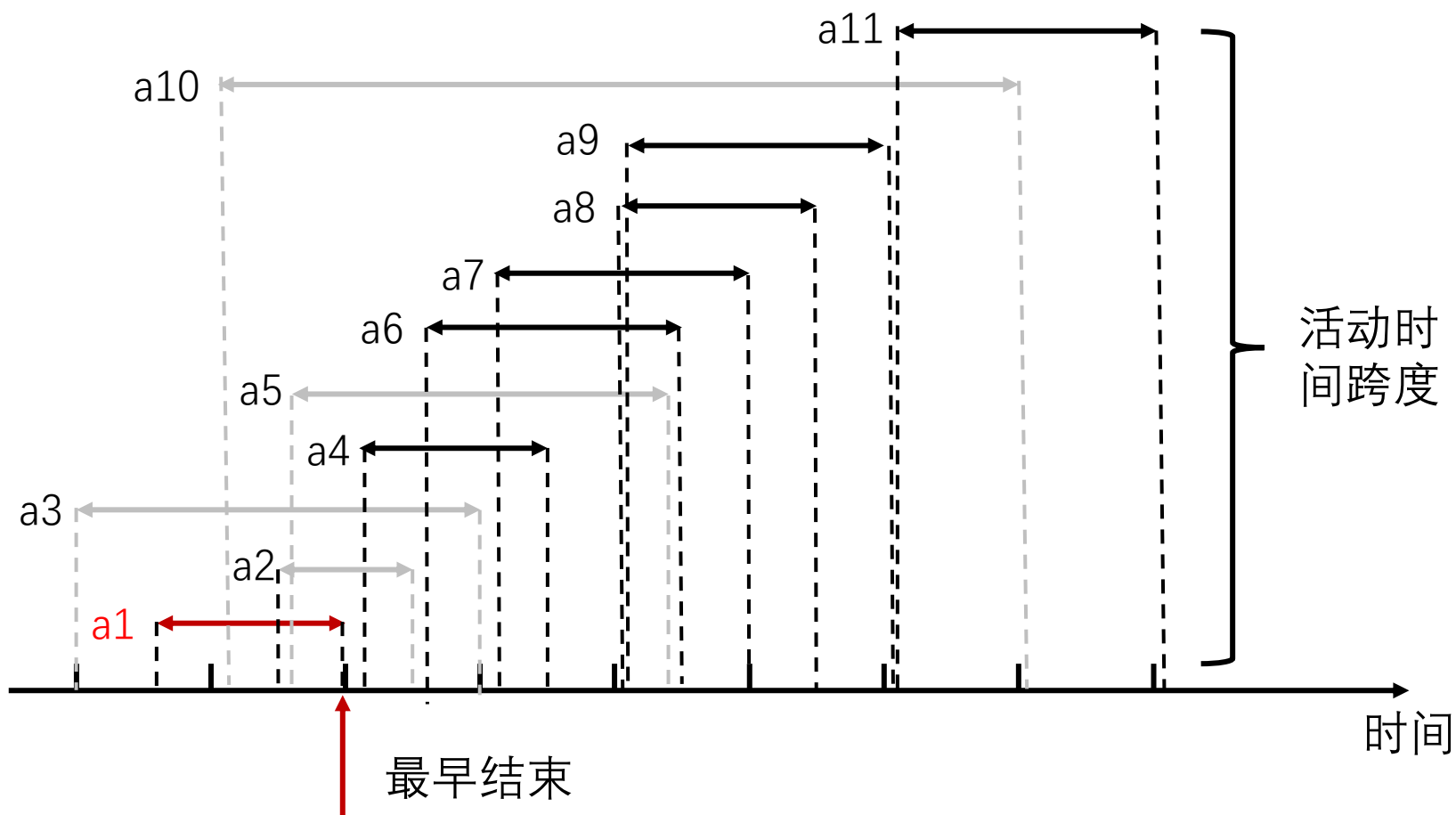
■ 算法实例



4.3 活动安排

■ 算法实例

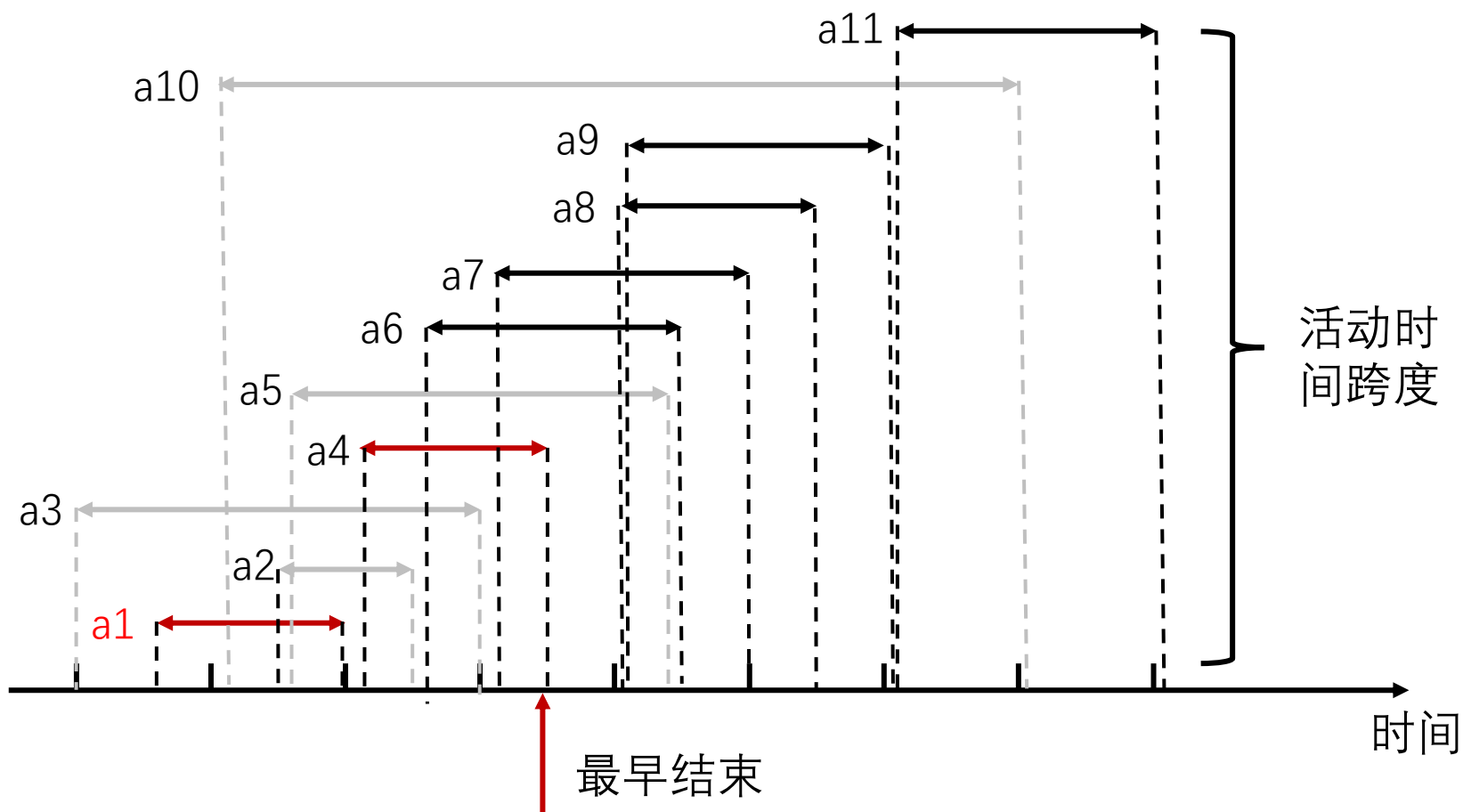
删除有冲突活动



4.3 活动安排

■ 算法实例

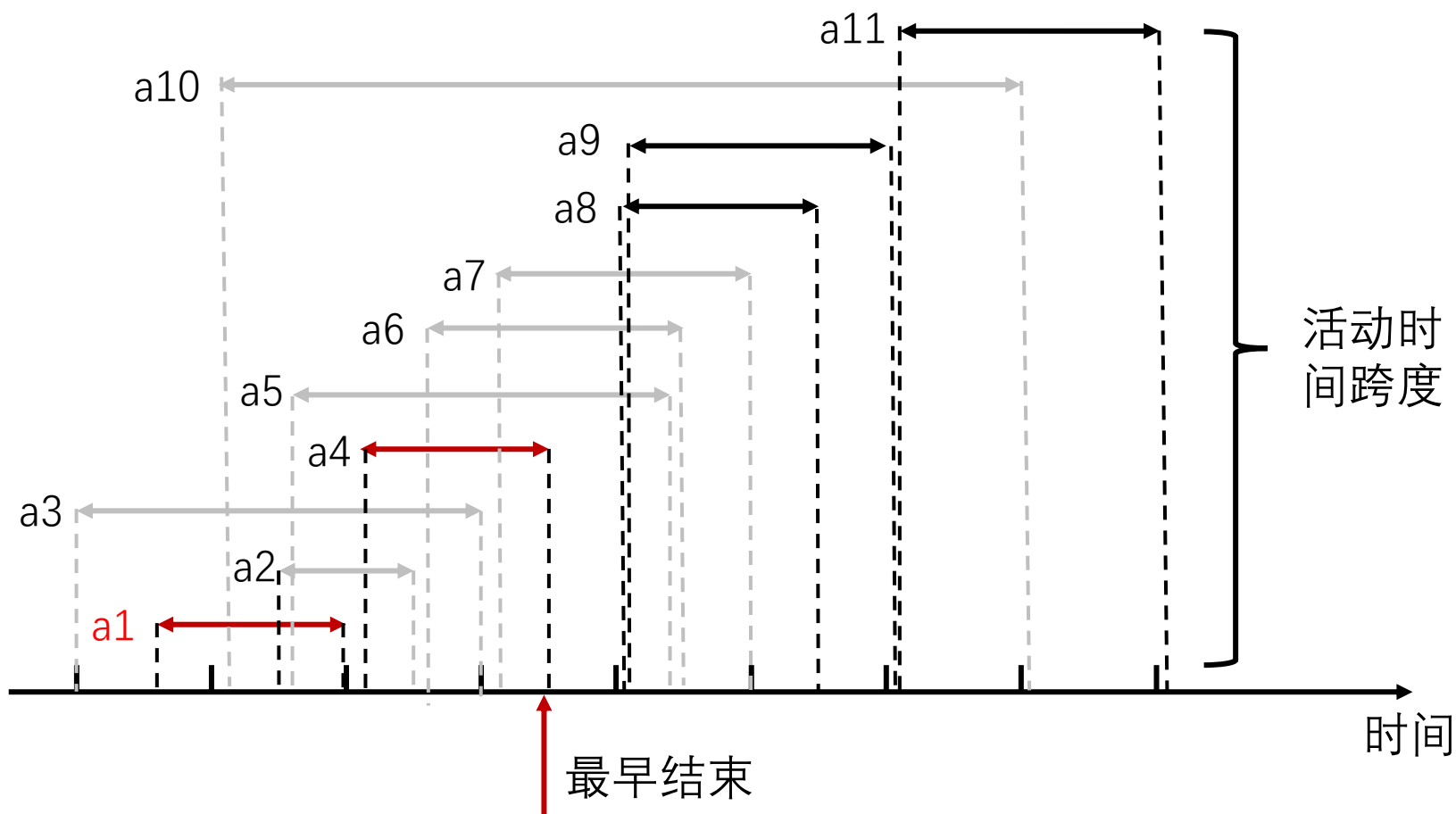
删除有冲突活动



4.3 活动安排

■ 算法实例

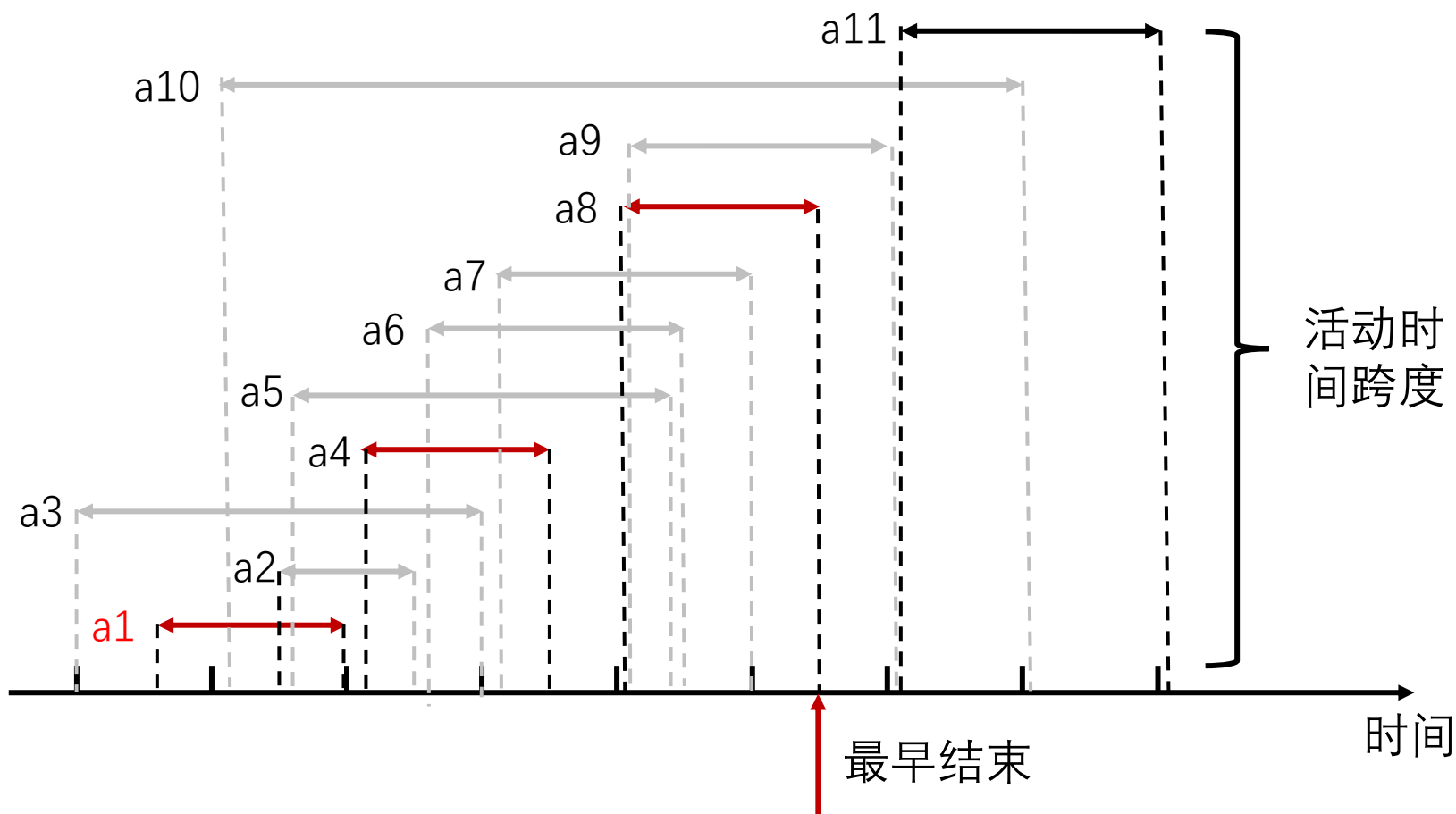
删除有冲突活动



4.3 活动安排

■ 算法实例

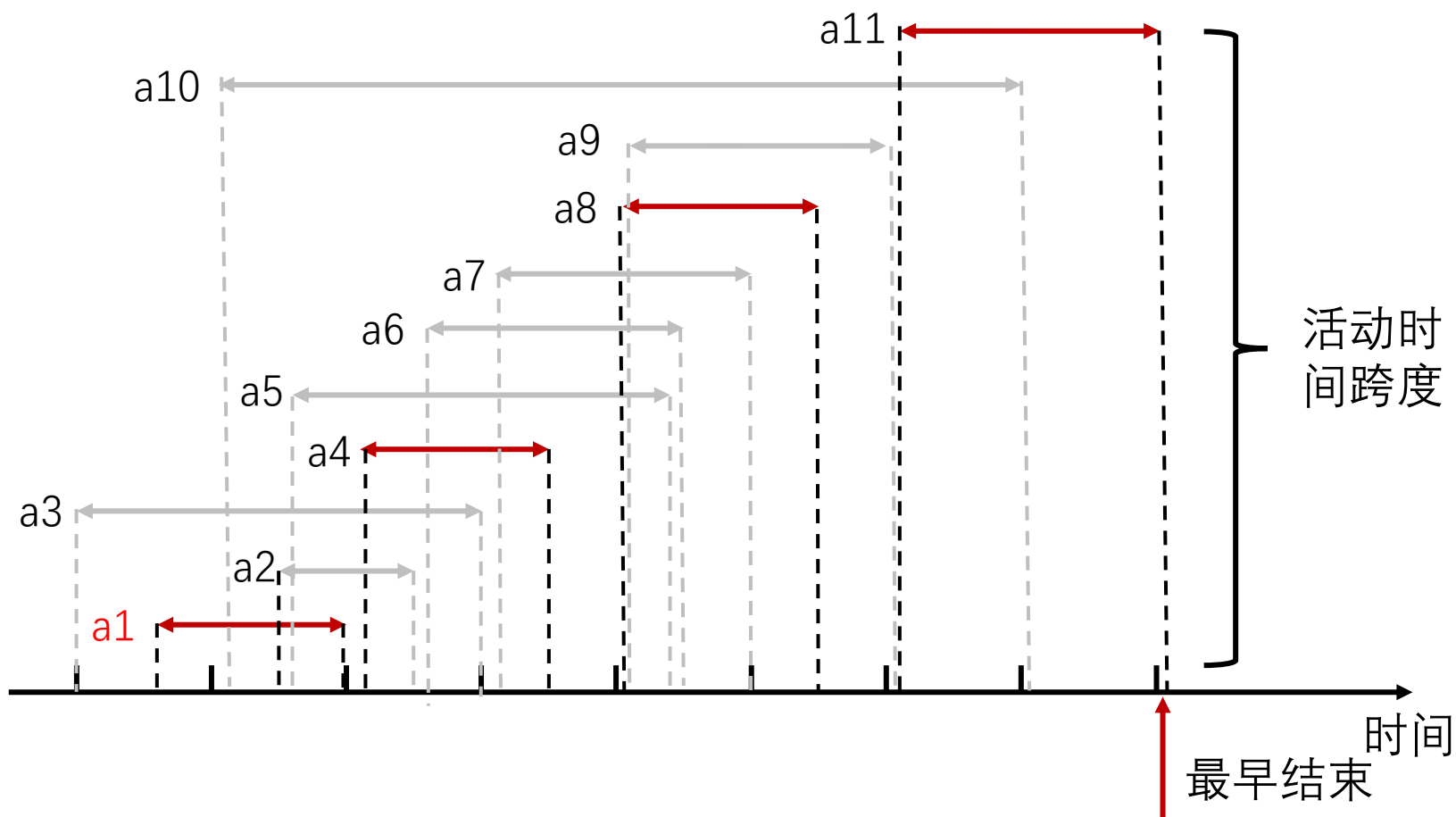
删除有冲突活动



4.3 活动安排

■ 算法实例

删除有冲突活动



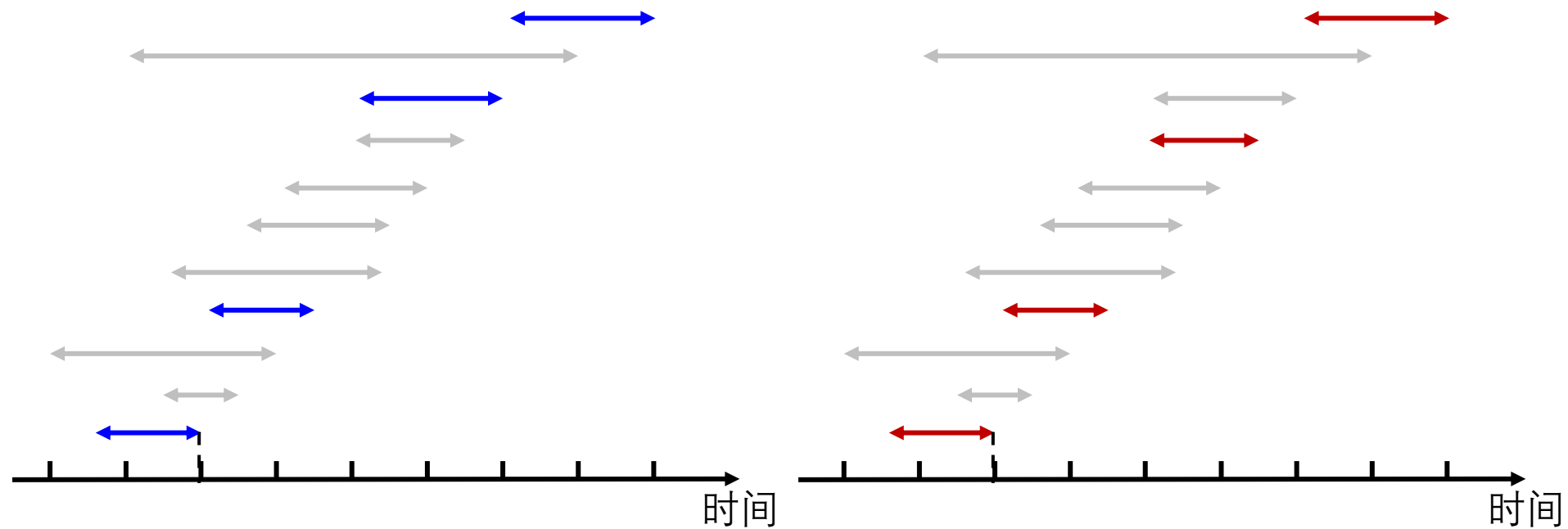
4.3 活动安排

- 正确策略，最早结束活动优先
- 证明：贪心解不劣于最优解

依次检查并替换

任意最优活动集合

贪心所得活动集合



4.3 活动安排

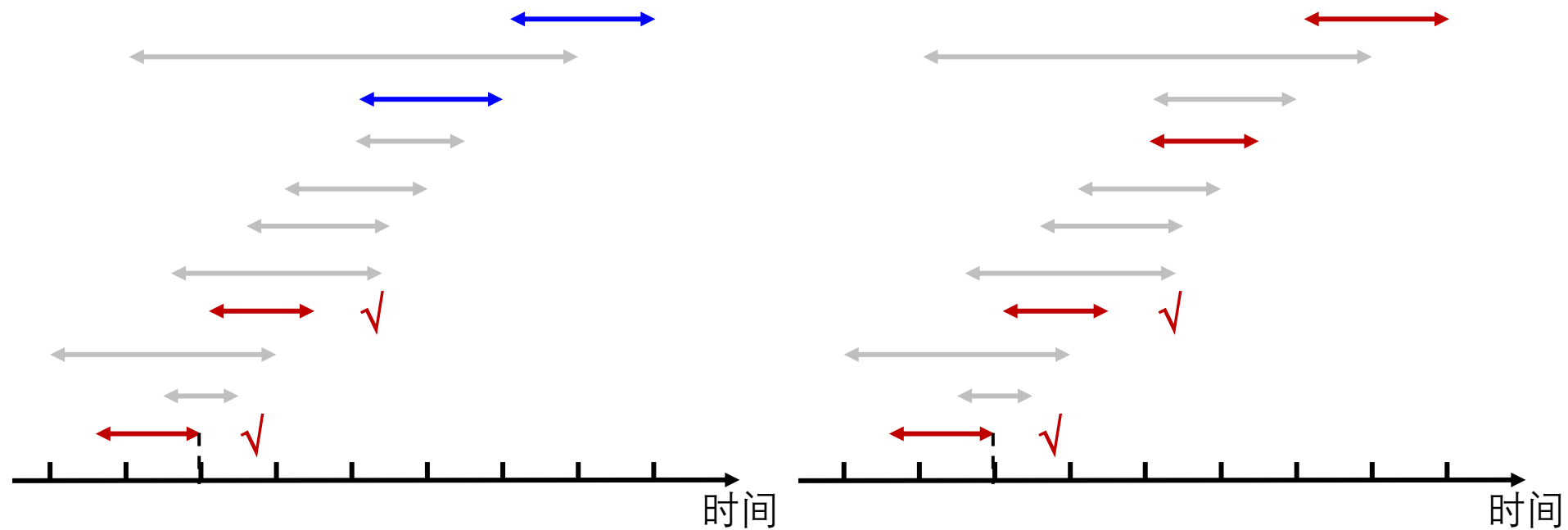
- 正确策略，最早结束活动优先
- 证明：贪心解不劣于最优解

任意最优活动集合



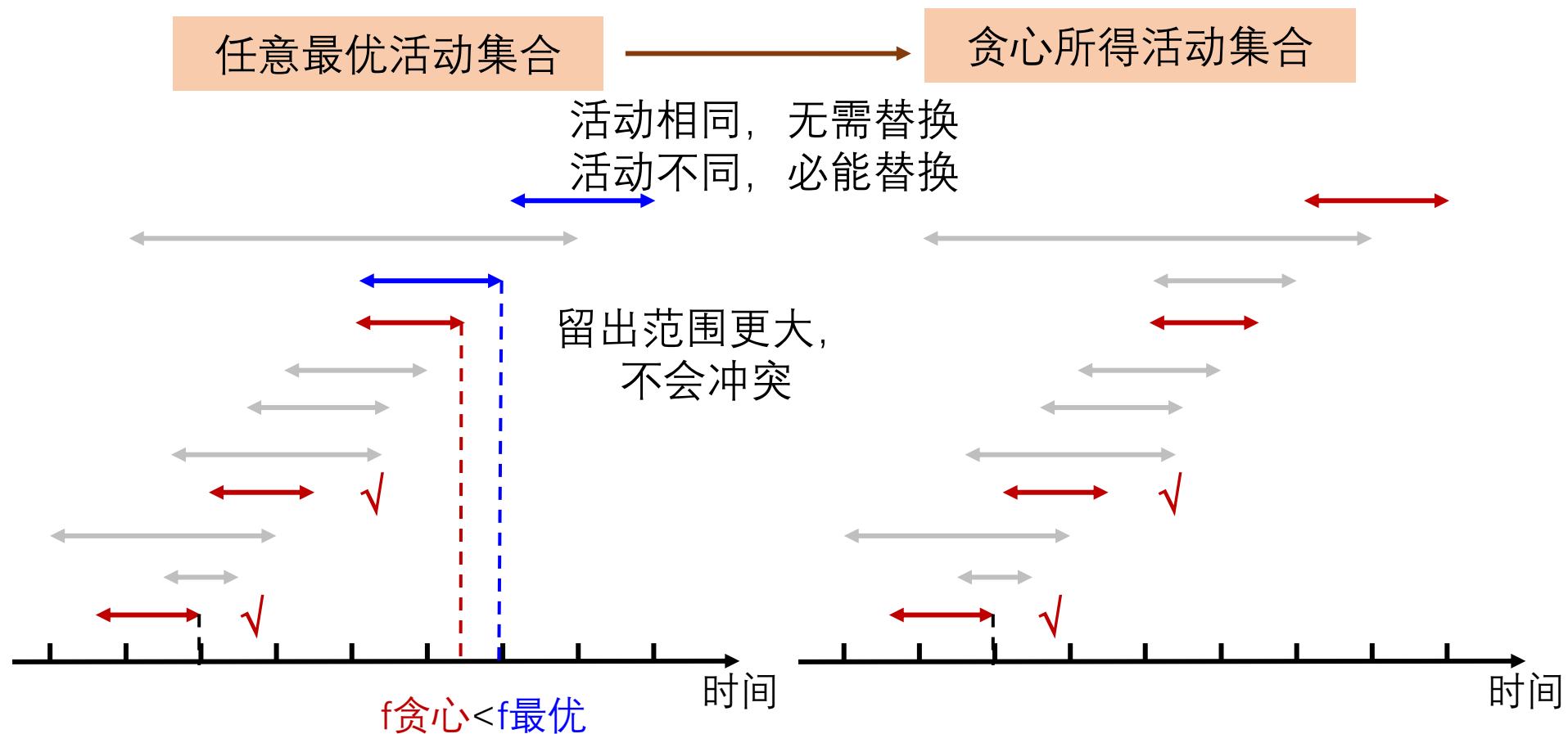
贪心所得活动集合

活动相同，无需替换



4.3 活动安排

- 正确策略，最早结束活动优先
- 证明：贪心解不劣于最优解



4.3 活动安排

- **输入:** n 个活动组成的集合 $A=\{a_1,a_2,\dots,a_n\}$, 每个活动 a_i 的开始时间 s_i 和结束时间 f_i
- **输出:** 找出活动集合 A 的子集 A' , 令 $\max|A'|$ | s.t. 所有 a_i,a_j 满足 $s_i \geq f_j$ 或 $s_j \geq f_i$
- 把活动按结束时间升序排序

- **复杂度分析** $O(n \log n)$

活动按结束时间升序排序 $n \log n$

顺序选择 n

```
int arrangeGreedy (int s[ ],int f[ ],bool a[ ])
{
    int n = s.length(),j = 1,i,count = 1;
    a[1] = true;//选择第1早结束的活动
    for(i = 2;i <= n;i++)
    {
        if(s[i] > f[j])
        {
            a[i] = true;//选择ai
            j = i;
            count++;
        }
        else
            a[i] = false;
    }
    return count;
}
```

实验4-2： 活动安排(贪心算法)

- 问题描述：输入 n 个活动组成的集合 $A=\{a_1, a_2, \dots, a_n\}$, 每个活动 a_i 的开始时间 s_i 和结束时间 f_i 。找出活动集合 A 的子集 A' , 令 $\max|A'|$ | s.t. 所有 a_i, a_j 满足 $s_i \geq f_j$ 或 $s_j \geq f_i$
- 示例 1:
输入: $s = \{1, 3, 0, 5, 3, 5, 6, 8, 8, 2, 12\}$;
 $f = \{4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14\}$;
输出:
■ 要求:
1 完成程序;
2 打印所选活动及活动数。

4.4 最优装载问题

■ **问题描述：** 假设有 n 个集装箱，其中集装箱 i 的重量为 w_i ，最优装载问题要求在装载体积不受限制的情况下，将尽可能多的集装箱装上载重量为 c 的轮船。

■ **形式化**

最优装载问题可描述为：

$$\max \sum_{i=1}^n x_i$$

$$\sum_{i=1}^n w_i x_i \leq c \quad x_i \in \{0,1\}, 1 \leq i \leq n$$

其中 $x_i = 1$ 表示装入集装箱 i , $x_i = 0$ 表示不装入集装箱 i 。

4.4 最优装载问题

■ 贪心策略：重量轻者优先

■ 证明

- **最优子结构的性质：** 设 $(x_1, x_2, \dots, x_n)$ 是最优装载问题满足贪心选择的最优解，则可知， $x_1=1$ ，且 $(x_2, \dots, x_n)$ 是轮船重量为 $c-w_1$ ，待装集装箱为 $\{2, 3, \dots, n\}$ 时相应最优装载问题的最优解。利用反证法证明。
- **贪心选择性质：** 设集装箱依重量从小到大排序， $(x_1, x_2, \dots, x_n)$ 是最优装载问题的一个最优解。设 $k = \min_{1 \leq i \leq n} \{i \mid x_i = 1\}$

$x: (0, 0, 1, 1, 1, 0, 0, 0) \quad k = 3$

$y: (1, 0, 0, 1, 1, 0, 0, 0)$

$$\sum_{i=1}^n w_i y_i = w_1 - w_k + \sum_{i=1}^n w_i x_i \leq \sum_{i=1}^n w_i x_i \leq c$$

4.4 最优装载问题

■ 实例

目标是装入集装箱个数最多——贪心选择重量轻者优先

例： $c=100$ 吨

$w_1=10$ 吨 $w_2=20$ 吨 $w_3=30$ 吨 $w_4=40$ 吨 $w_5=45$ 吨

① 初始剩余载重 $g=100$ 装入个数 $k=0$

② 贪心选择 w_1

$w_1 < g$ 故 $x_1=1$ $g=g-w_1=90$ $k=1$

③ 贪心选择 w_2

$w_2 < g$ 故 $x_2=1$ $g=g-w_2=70$ $k=2$

同理，继续。得 $X=\{1, 1, 1, 1\}$

4.4 最优装载问题

■ 算法

```
typedef struct {
    float w;
    int i;
} Item;

float loading(float c, float w[], int x[], int n) {
    Item d[n];
    for (int i = 0; i < n; i++) d[i].w = (w[i], i);
    mergeSort(d);          // 排序 O(nlogn)
    float opt = 0;
    for (int i = 0; i < n; i++) x[i] = 0;

    for (int i = 0; i < n && d[i].w <= c; i++) { // 贪心选择
        x[d[i].i] = 1;
        opt += d[i].w;
        c -= d[i].w;
    }
    return opt;
}
```

时间复杂性： 排序 $O(n \log n)$ ，贪心选择 $O(n)$ ，时间复杂性为 $O(n \log n)$

4.5 最大子段和

- 问题描述：给定n个数(可以为负数)的序列 $(a_1, a_2, \dots, a_n)$ ，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

$$\max\{0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k\}$$

- 实例1

输入：(-2, 11, -4, 13, -5, -2)

输出：最大子段和为 $a_2 + a_3 + a_4 = 20$

- 实例2

输入：[-2, 1, -3, 4, -1, 2, 1, -5, 4]

输出：6

解释：连续子数组 [4, -1, 2, 1] 的和最大，为 6。

4.5 最大子段和

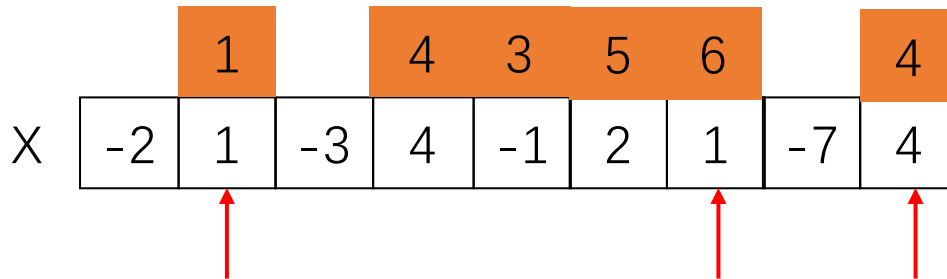
- 算法1：蛮力，对所有的 (i, j) 对，顺序求和 $a_i + \dots + a_j$ 并比较出最大的和。
- 算法2：分治策略，将数组分成左右两半，分别计算左边的最大和、右边的最大和、跨边界的最大和，然后比较其中最大者。
- 算法3：动态规划
- 算法4：贪心算法

4.5 最大子段和 贪心算法

- 贪心思想

- 局部最优：当前“连续和”为负数的时候立刻放弃，从下一个元素重新计算“连续和”，因为负数加上下一个元素“连续和”只会越来越小。
- 全局最优：选取最大“连续和”。

- $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$



4.5 最大子段和 贪心算法

■ 算法

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int result = 0;
        int sum = 0;
        for (int i = 0; i < nums.size(); i++) {
            sum += nums[i];
            if (sum > result) { // 取区间累计的最大值（不断确定最大子序终止位置）
                result = sum;
            }
            if (sum <= 0) sum = 0; // 相当于重置最大子序起始位置，因为遇到负数一定是
拉低总和
        }
        return result;
    }
};
```

实验4-3：最大子段和(贪心算法)

- 问题描述：给定 n 个数(可以为负数)的序列 $(a_1, a_2, \dots, a_n)$ ，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。
- 示例 1：
输入: $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$
输出: 6
解释: 连续子数组 $[4, -1, 2, 1]$ 的和最大，为 6。
- 要求：
 - 1 完成程序；
 - 2 打印子段范围。

4.6 多机调度问题

- **问题描述**：多机调度问题要求给出一种作业调度方案，使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。
- **约定**：每个作业均可在任何一台机器上加工处理，但未完工前不允许中断处理。作业不能拆分成更小的子作业。
- **NP完全问题**，到目前为止还没有有效的解法。对于这一类问题,用贪心选择策略有时可以设计出较好的近似算法。
- **贪心策略**：贪心选择作业执行时间大者，将其调度到可最早开始加工它的机器上。此贪心选择策略可以设计出解多机调度问题的较好的近似算法。

4.6 多机调度问题

■ 实例：假设7个独立作业{1,2,3,4,5,6,7}由3台机器M1,M2和M3加工处理。

i	1	2	3	4	5	6	7
t_i	7	3	4	5	3	8	3

设分别用 $T[3]$ 表示3台机器的最早开始时间。

初始： $T[0]=T[1]=T[2]=0$

作业排序得： 6 1 4 3 2 5 7

1. 选作业6，调度至机器0， $T[0]=8$
2. 选作业1，调度至机器1， $T[1]=7$
3. 选作业4，调度至机器2， $T[2]=5$
4. 选作业3，调度至机器2， $T[2]=9$
5. 选作业2，调度至机器1， $T[1]=10$
6. 选作业5，调度至机器0， $T[0]=11$
7. 选作业7，调度至机器2， $T[2]=12$

多机问题不一定得到最优解：不具有贪心选择性质和最优子结构。
多机调度是NP完全问题，贪心算法得到其近似解。

P_0	6 (8)		5 (3)		
P_1	1 (7)		2 (3)		
P_2	4 (5)		3 (4)		7 (3)

4.6 多机调度问题

算法

1. 将数组 $t[n]$ 由大到小排序, 对应的作业序号存储在数组 $p[n]$ 中;
2. 将数组 $d[m]$ 初始化为0;
3. for ($i=1; i \leq m; i++$)
 - 3.1 $S[i]=\{p[i]\}$; //将 m 个作业分配给 m 个机器
 - 3.2 $d[i]=t[i]$;
4. for ($i=m+1; i \leq n; i++$)
 - 4.1 j =数组 $d[m]$ 中最小值对应的下标; //为最先空闲的机器序号
 - 4.2 $S[j]=S[j]+\{p[i]\}$; //将作业 i 分配给最先空闲的机器 j
 - 4.3 $d[j]=d[j]+t[i]$; //机器 j 将在 $d[j]$ 后空闲

当 $n \leq m$ 时, 只要将机器 i 的 $[0, t_i]$ 时间区间分配给作业 i 即可, 算法只需要 $O(1)$ 时间。

当 $n > m$ 时, 首先将 n 个作业依其所需的处理时间从大到小排序。然后依此顺序将作业分配给空闲的处理机。算法所需的计算时间为 $O(n \log n)$ 。

4.7 货郎担问题

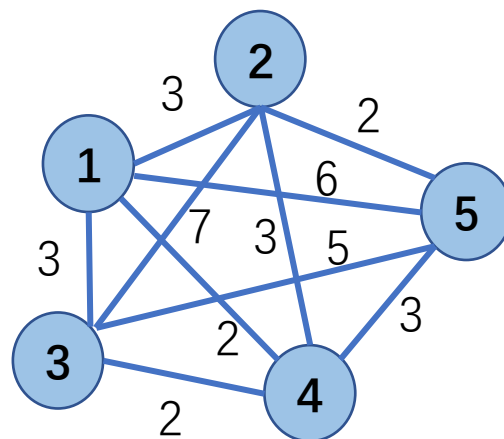
■问题原型

某售货员要到若干个城市销售货物，已知各城市之间的距离，要求售货员选择出发的城市及旅行路线，使每个城市经过一次，最后回到原出发城市，而总路程最短。

4.7 货郎担问题

数学描述

假设 n 个城市，分别用 $1 \sim n$ 的数字编号。货郎担问题是在有向网 $G = \langle V, E \rangle$ ， $V = \{1, 2, \dots, n\}$, $E = \{e_{ij}, 1 \leq i, j \leq n\}$ (e_{ij} 上权表示城市 i 到城市 j 的距离) 中寻找一条路径最短的哈密尔顿回路（经过每个顶点一次）问题。



4.7 货郎担问题

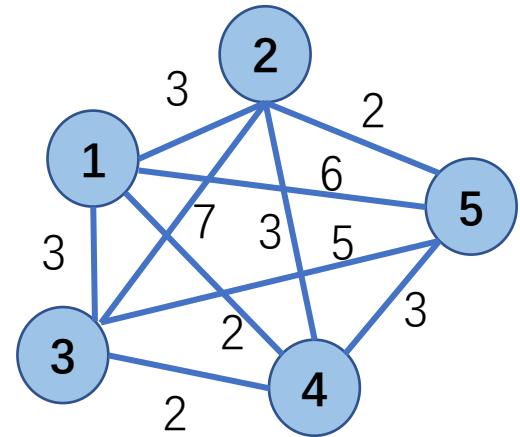
贪心法解货郎担问题

求距离最短的哈密尔顿回路，需求从各城市出发的最短回路，再对 n 条回路求极小值。

回路最短—贪心选择距离短的路线

求解货郎担问题的两种贪心策略：

(1) **最近邻点策略**：从任意城市出发，每次在没有到过的城市中选择最近的一个，直到经过了所有的城市，最后回到出发城市。

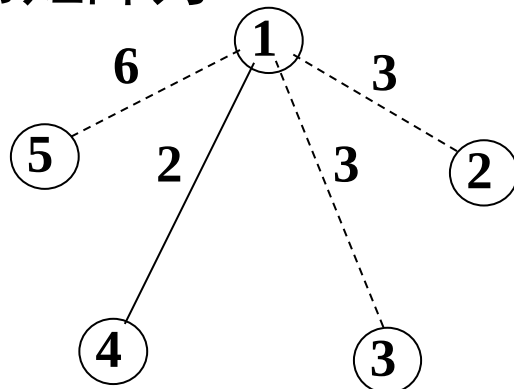


4.7 货郎担问题

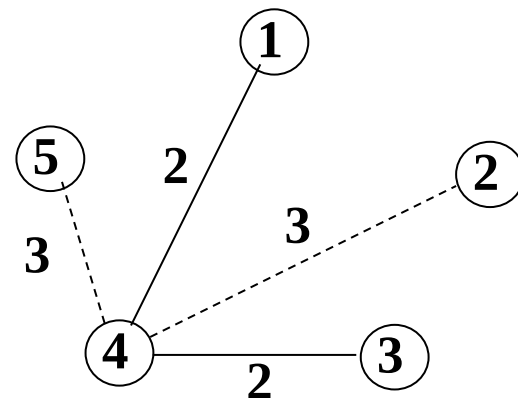
例：假设5个城市费用矩阵为：最近邻点贪心策略求解货郎担问题的过程

$$C = \begin{pmatrix} \infty & 3 & 3 & 2 & 6 \\ 3 & \infty & 7 & 3 & 2 \\ 3 & 7 & \infty & 2 & 5 \\ 2 & 3 & 2 & \infty & 3 \\ 6 & 2 & 5 & 3 & \infty \end{pmatrix}$$

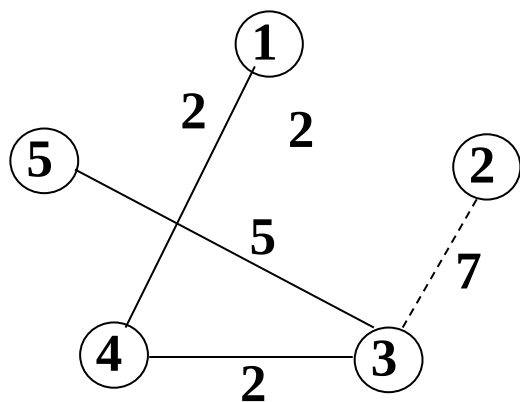
(a) 5城市的代价矩阵



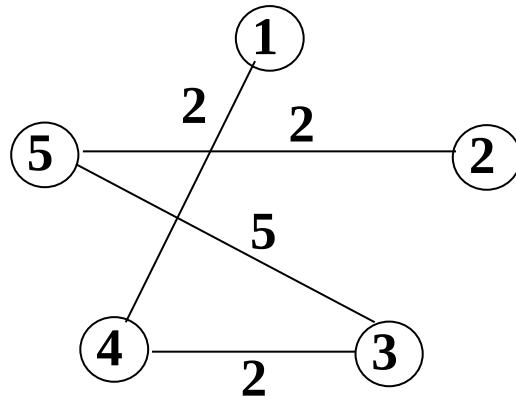
(b) 城市1→城市4



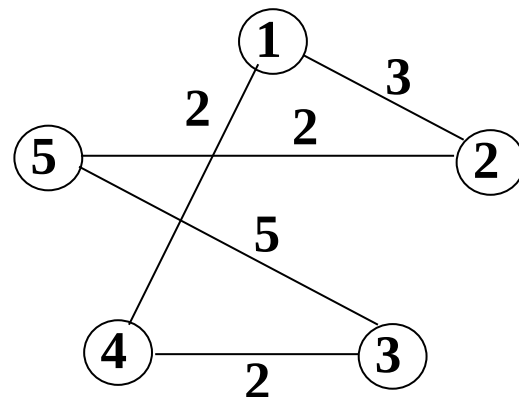
(c) 城市4→城市3



(d) 城市3→城市5



(e) 城市5→城市2



(f) 城市2→城市1

从城市1出发：1→4→3→5→2→1，总距离14

从城市2出发：2→5→4→1→3→2，总距离17

其它城市同理，再在5条回路中选择最小距离。

4.7 货郎担问题

设图 G 有 n 个顶点，边上的代价存储在二维数组 $w[n][n]$ 中，集合 V 存储图的顶点，集合 P 存储经过的边，最近邻点策略求解货郎担问题的算法如下：

算法1——最近邻点策略求解货郎担问题

- 1 . $P = \{ \}$;
- 2 . $V = V - \{u_0\}$; $u = u_0$; //从顶点 u_0 出发
- 3 . 循环直到集合 P 中包含 $n-1$ 条边
 - 3.1 查找与顶点 u 邻接的最小代价边 (u, v) 并且 v 属于集合 V ;
 - 3.2 $P = P + \{(u, v)\}$;
 - 3.3 $V = V - \{v\}$;
 - 3.4 $u = v$; //从顶点 v 出发继续求解

4.7 货郎担问题

贪心法解货郎担问题算法分析

时间复杂性分析： 从一个城市出发的最短回路计算为 $O(n^2)$ ，共 n 个城市，故时间复杂度为 $O(n^3)$ ，远小于穷举法的 $n!$ 。

最优解分析：

从城市1出发最优路线：1→2→5→4→3→1，距离13

贪心法求解货郎担问题不具有贪心选择性质和最优子结构性质。只能得到近似解，不能得到最优解。

4.7 货郎担问题

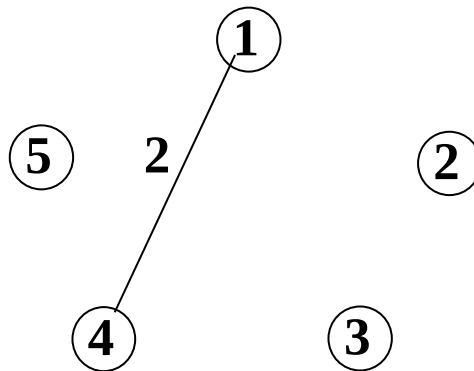
(2) **最短链接**策略：每次在整个图的范围内选择最短边加入到解集合中，但是，要保证加入解集合中的边最终形成一个哈密顿回路。因此，当从剩余边集 E' 中选择一条边 (u, v) 加入解集合 S 中，应满足以下条件：

- ① 边 (u, v) 是边集 E' 中代价最小的边；
- ② 边 (u, v) 加入解集合 S 后， S 中不产生回路；
- ③ 边 (u, v) 加入解集合 S 后， S 中不产生分枝；

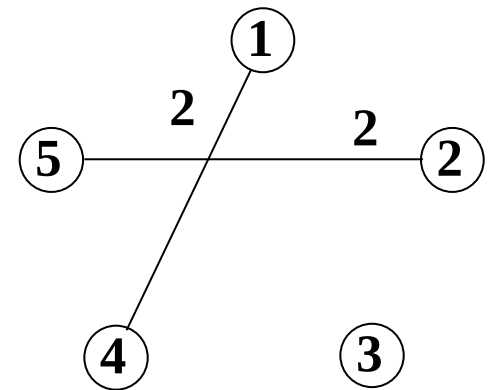
4.7 货郎担问题

$$C = \begin{pmatrix} \infty & 3 & 3 & 2 & 6 \\ 3 & \infty & 7 & 3 & 2 \\ 3 & 7 & \infty & 2 & 5 \\ 2 & 3 & 2 & \infty & 3 \\ 6 & 2 & 5 & 3 & \infty \end{pmatrix}$$

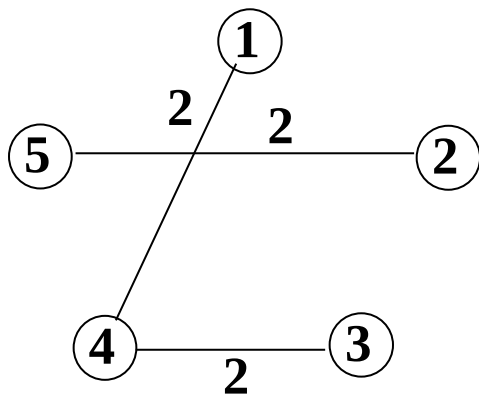
(a) 5城市的代价矩阵



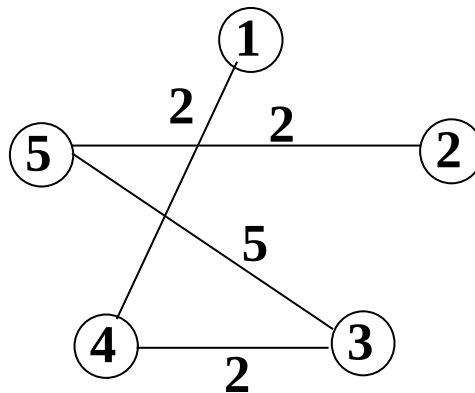
(b) 城市1→城市4



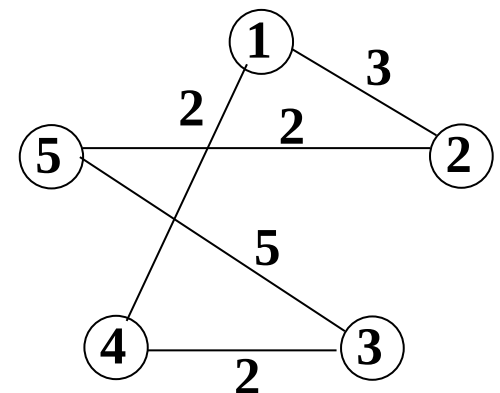
(c) 城市5→城市2



(d) 城市4→城市3



(e) 城市3→城市5



(f) 城市2回到出发城市1

最短链接贪心策略求解货郎担问题的过程

4.7 货郎担问题

设图 G 有 n 个顶点，边上的代价存储在二维数组 $w[n][n]$ 中，集合 E' 是候选集合即存储所有未选取的边，集合 P 存储经过的边，最短链接策略求解货郎担问题的算法如下：

算法2——最短链接策略求解货郎担问题

- 1 . $P = \{ \}$;
- 2 . $E' = E$; //候选集合，初始时为图中所有边
- 3 . 循环直到集合 P 中包含 $n-1$ 条边
 - 3.1 在 E' 中选取最短边 (u, v) ;
 - 3.2 $E' = E' - \{(u, v)\}$;
 - 3.3 如果 (顶点 u 和 v 在 P 中不连通 and 不产生分枝)
则 $P = P + \{(u, v)\}$;

4.8 数据结构中的贪心法

- 单源点最短路径
- 最小生成树 — prim和Kruskal
- Huffman编码

4.9 贪心算法应用小结

比较动态规划与贪心算法都是求解最优化问题：

1. 着眼点不同

动态规划算法着眼全局，而贪心算法着眼局部。

2. 求解的结果可能不同

动态规划算法求解结果总是最优的，贪心算法对大多数优化问题能得到最优解，但有时并不能求得最优解。

3. 求解效率上的差异

从求解效率来说，贪心算法比动态规划更高，且一般不存在空间限制的影响。

4. 求解范围上的差异

应用贪心算法有时可简化一些构造性问题。